

Problem Set

Task:

Build and compare two routing strategies: **#1 Fine-Tuning BERT & its variants**, and **#2 Fine-Tuning LLM/SLM**, to automatically classify user intent and dispatch it to the right specialized agent.

Context: You are part of the AI engineering team at **ShopAssist AI**, a company developing a next-generation agentic customer support platform for e-commerce businesses. The platform replaces a traditional support inbox with a fleet of specialized AI agents, each trained to handle a specific category of customer problems.

The first step that must occur when a customer sends a message is routing: the system reads the message and determines which agent should handle it. Get the routing wrong, and a customer asking about a missing parcel ends up talking to the billing agent, who cannot help, leading to a frustrating hand-off and a lost customer.

Your team has been tasked with prototyping **two candidate routing approaches**, benchmarking them, and providing a recommendation using real customer support data. The constraint is that the solution must be accurate enough to trust in production and light enough to run cost-efficiently at volume.

What the incoming messages look like

- "I placed an order yesterday, but I haven't received any confirmation email. Can you check if it went through?"
- "My card was charged twice for the same order. I need a refund immediately."

Each message must be routed to exactly one agent. The routing decision happens before any agent reads the message, so it must be fast, consistent, and correct, even when the customer's phrasing is informal, abbreviated, or ambiguous.

THE TEN SPECIALIZED AGENTS:

1. **ACCOUNT:** create_account, delete_account, edit_account, switch_account
2. **CANCELLATION_FEE:** check_cancellation_fee
3. **DELIVERY:** delivery_options
4. **FEEDBACK:** complaint, review
5. **INVOICE:** check_invoice, get_invoice
6. **NEWSLETTER:** newsletter_subscription

7. **ORDER:** cancel_order, change_order, place_order
8. **PAYMENT:** check_payment_methods, payment_issue
9. **REFUND:** check_refund_policy, track_refund
10. **SHIPPING_ADDRESS:** change_shipping_address, set_up_shipping_address

Dataset: Bitext Customer Support LLM Chatbot Training Dataset

HuggingFace: [bitext/Bitext-customer-support-llm-chatbot-training-dataset](#)

- **26,872** real-phrasing customer support queries covering 27 intents across **10 categories**. Each row has an instruction (the customer message), a category, and an intent field. Use the **category** field directly as your classification target, i.e., the 10 categories are the 10 agents above.
- **Splits (stratified by category, random_seed = 0):**
 - **95% train · 10% validation · 10% test.** Both approaches are evaluated on the same held-out test set. Do not use the test set to make any modelling decisions.

Evaluation Metrics:

All approaches are evaluated on: **precision, recall, macro-F1 (primary metric), and accuracy**. Additionally, report required **GPU memory** and **inference latency**.

Tasks:

1. Approach 1: Fine-Tuning Encoder-Only Transformers

- Fine-tune the mentioned **encoder-only transformers** as a 10-class classifier on the training split. Add a linear classification head on top of the [CLS] token, train using AdamW with a linear warmup scheduler, and evaluate on the held-out test set. Report all required metrics, training and validation loss curves, a confusion matrix, and inference latency. Log peak GPU memory during inference.
- **Candidate Models:** [google-bert/bert-base-uncased](#), [distilbert/distilbert-base-uncased](#), [FacebookAI/roberta-base](#), [answerdotai/ModernBERT-base](#)

2. Approach 2: Fine-Tuning Decoder-Only Transformers (with/without LoRA)

- Fine-tune **SLM**, aka Small language models (**LLMs with fewer PARAMS**) for routing by treating classification as a text generation problem. Format each training example as an instruction-input pair where the model's target output is simply the agent name as a single token, for example, **REFUND** or **ORDER**. Evaluate on the held-out test set, and report all required metrics, confusion matrix, inference latency, and peak GPU memory.

- **Candidate Models:** [Qwen/Qwen3.5-0.8B](#), [Qwen/Qwen3.5-2B](#), [Qwen/Qwen3.5-4B](#)
- **GPU MEMORY NOTE:**
 - If you hit GPU memory limits on a free-tier GPU (Colab T4, ~15 GB), consider using LoRA/QLoRA via the peft library.

Note: You are free to experiment with any models, optimizers, and other training configurations for Approach 1 and Approach 2 and report the best one in the final comparison table. Best is determined by the highest macro-F1 on the test set.

3. Comparison + Recommendation + Reflection

- **Comparison Criteria:**
 - **Test Set:** Precision, Recall, Macro-F1, Accuracy, GPU Memory, and Inference Latency.
- **Recommendation:**
 - Write one paragraph (150–200 words) addressed to your engineering lead at ShopAssist AI. State which approach you would deploy, cite at least three specific numbers from your experiments, name the main production risk of your chosen approach, and describe how you would monitor for it.
- **Reflection:**
 - **1: On what transfers from pretraining**
 - Both Encoder-Only and Decoder-Only models started with pretrained weights trained on general text. Neither has ever seen a customer support ticket. What exactly did pretraining contribute to this task, and what did the fine-tuning have to teach from scratch?
 - **2: On Handling Chit-Chat Queries**
 - A customer sends: “Hey! How’s it going?”, a message with no chat routing intent at all. Walk through how each of your approaches handles this. Which one fails most gracefully? What would you add to the system to handle this class of input in a production environment?

Submission Requirements:

1. **Single file support_routing.ipynb:**
 - all code for approaches #1, #2, and the comparison. Must run top-to-bottom without errors on a fresh kernel.
2. **Required Plots** (All in line):

- loss curves, confusion matrix, and comparison bar chart.
- 3. **COMPARISON:**
 - Completed comparison table in a dedicated cell with your measured values before submission.
- 4. **RECOMMENDATION + REFLECTION:**
 - Separate markdown cell for each.
 - **Recommendation cell** must follow the provided guidelines.
 - **Reflection cell** must answer all the reflection questions.

References:

- **BERT Fine-Tuning:**
 - **HuggingFace – FINE-TUNING A PRETRAINED MODEL:**
 - [HuggingFace – Processing the data](#)
 - [HuggingFace – Fine-Tuning a model with the Trainer API](#)
 - [HuggingFace – Write your training loop in PyTorch](#)
 - [HuggingFace — Text Classification Guide](#) (*hands-on guide, ~20 min — actually run the code*)
 - [HuggingFace – Understanding and Interpreting Learning Curves](#)
 - [Jay Alammar — A Visual Guide to Using BERT for the First Time](#) (*article, ~20 min*)
- **LLM Fine-Tuning:**
 - [DeepLearning.AI — Fine-Tuning Large Language Models](#) (*short course, 1h35m, Beginner-Friendly*)
 - [FreeCodeCamp — Fine-Tuning LLM Models](#) (*long-form video course, 2h 35m, Intermediate*)
 - **Assignment References:**
 - [Fine-Tuning Llama 3.1 for Text Classification | DataCamp](#) (*tutorial, ~45 min*)
 - [LLM as a Router: How to Fine-Tune Models for Intent-Based Workflows](#) (*article, ~15 min — directly relevant to your upcoming assignment*)
 - **Hugging Face (LLM Course)**
 - [11. FINE-TUNE LARGE LANGUAGE MODELS](#)